

Conceito e Boas Práticas de Listas

- **Ordenação lógica:** Organize por ordem alfabética ou numérica.
- **Objetividade:** Mantenha os itens curtos e fáceis de visualizar.
- **Consistência:** Mantenha ícones, textos e ações com formato padronizado.
- **Estilos:** Podem ser apresentadas no estilo **Padrão** ou **Segmentado**.

Anatomia de um Item de Lista

O Jetpack Compose organiza um `ListItem` utilizando diversas seções configuráveis:

1. **Container:** A área/caixa que engloba o item (Obrigatório).
2. **Headline (headlineContent):** O texto do título principal do item (Obrigatório).
3. **Overline (overlineContent):** Um pequeno texto posicionado acima do título principal.
4. **Supporting text (supportingContent):** Texto secundário com informações adicionais.
5. **Leading Content (leadingContent):** Elemento exibido no início do item (pode ser um Ícone, Avatar ou Mídia/Imagem).
6. **Trailing Content (trailingContent):** Elemento exibido no final do item (como um ícone, texto meta ou contadores).
7. **Divider (HorizontalDivider):** Linha separadora horizontal entre os itens da lista.

Exemplo 1 – Lista Simples de Uma Linha

```
@Composable
fun UmaLinhaSimples() {
    Column {
        HorizontalDivider()
        ListItem(
            headlineContent = { Text("One line list item with 24x24 icon") },
            leadingContent = {
                Icon(Icons.Filled.Favorite, contentDescription = "Localized description 1")
            },
        )
        HorizontalDivider()
    }
}
```

O que faz: Desenha uma linha divisória, cria um `ListItem` com um ícone de coração

(leadingContent) e um texto simples (headlineContent), e fecha com outra linha divisória.

Exemplo 2 – Lista Simples de Duas Linhas

```
@Composable
fun DuasLinhasSimples(){
    Column {
        HorizontalDivider()
        ListItem(
            headlineContent = { Text("Two line list item with trailing") },
            supportingContent = { Text("Secondary text") },
            trailingContent = { Text("meta") },
            leadingContent = {
                Icon(Icons.Filled.Favorite, contentDescription = "Localized description")
            },
        )
        HorizontalDivider()
    }
}
```

O que faz: Além do ícone inicial e do título principal, inclui um texto explicativo secundário (supportingContent) e uma informação final meta à direita (trailingContent).

Exemplo 3 – Lista de Três Linhas Completa

```
@Composable
fun TreslinhasComSuporte(){
    Column {
        HorizontalDivider()
        ListItem(
            headlineContent = { Text("Three line list item") },
            overlineContent = { Text("OVERLINE") },
            supportingContent = { Text("Secondary text") },
            leadingContent = {
                Icon(Icons.Filled.Favorite, contentDescription = "Localized description")
            },
            trailingContent = { Text("meta") },
        )
        HorizontalDivider()
    }
}
```

O que faz: Adiciona a propriedade overlineContent, posicionando a palavra "OVERLINE" em destaque acima do texto principal.

Exemplo 4 – Lista Interativa com Clique Simples

```
@Composable
fun ItemsComClique() {
    // Cria uma lista observável contendo 3 contadores iniciados em 0
    val counts = remember { mutableStateListOf(0, 0, 0) }

    Column {
        HorizontalDivider()

        repeat(3) { idx ->
            ListItem(
                // Ao clicar no item, incrementa o contador daquele índice específico
                modifier = Modifier.clickable { counts[idx]++ },
                leadingContent = { Icon(Icons.Default.Home, contentDescription = null) },
                trailingContent = { Text("${counts[idx]}") },
                supportingContent = { Text("Additional info") },
                headlineContent = { Text("Item ${idx + 1}") },
            )
            HorizontalDivider()
        }
    }
}
```

O que faz: A função `repeat(3)` gera três itens numerados. O modificador `Modifier.clickable` atualiza o estado `counts[idx]`, fazendo com que o `trailingContent` seja renderizado dinamicamente com o número atualizado de cliques.

Exemplo 5 – Alternância de Modos com Clique Longo

```
@Composable
fun ItensComCliqueLongo() {
    Column {
        HorizontalDivider()

        // Estado do modo atual: true = Modo de Contagem / false = Modo de Seleção
        var inClickMode by remember { mutableStateOf(true) }
        val counts = remember { mutableStateListOf(0, 0, 0) }
        val checked = remember { mutableStateListOf(false, false, false) }

        repeat(3) { idx ->
            if (inClickMode) {
                // Modo 1: Incrementa contador com clique e muda de modo com clique longo
                ListItem(
                    modifier = Modifier.combinedClickable(
                        onClick = { counts[idx]++ },
                        onLongClick = {
                            checked[idx] = true
                            inClickMode = false
                        }
                    ),
                    leadingContent = { Icon(Icons.Default.Home, contentDescription = null) },
                    trailingContent = { Text("${counts[idx]}") },
                    supportingContent = { Text("Long-click to change interaction mode.") },
                    headlineContent = { Text("Item ${idx + 1}") },
                )
            } else {
                // Modo 2: Marcar/desmarcar Checkbox e retorna ao modo 1 com clique longo
                ListItem(
                    modifier = Modifier.combinedClickable(
                        onClick = { checked[idx] = !checked[idx] },
                        onLongClick = {
                            inClickMode = true
                            checked.replaceAll { false }
                        }
                    ),
                    leadingContent = {
                        Checkbox(
                            checked = checked[idx],
                            onCheckedChange = null
                        )
                    },
                    trailingContent = { Icon(Icons.Default.Favorite, contentDescription = null) },
                    supportingContent = { Text("Long-click to change interaction mode.") },
                    headlineContent = { Text("Item ${idx + 1}") },
                )
            }
        }

        HorizontalDivider()
    }
}
```

O que faz:

1. No Modo Contagem (`inClickMode = true`), um clique simples incrementa o contador e um clique longo ativa a caixa de seleção e altera `inClickMode` para `false`.
KT
2. No Modo Seleção (`inClickMode = false`), a lista passa a exibir um `Checkbox` no `leadingContent`. Clicar no item altera o valor booleano (`checked[idx]`), e fazer um clique longo reseta as seleções e volta ao modo inicial.

